

SQnic: a smaller path to better memory

research and implementation priorities | 5 september 2026

for Caleb • scope: local cross-harness conversation, tool and Git context

recommendation

keep Rust and SQLite. the next release should improve evidence selection, state tracking and handoff quality. the current measurements do not show a need to replace the storage engine. they do show slower broad searches and refreshes as history grows.

build a source-preserving memory layer: immutable events, small groups of related events, explicit current-state records, and a bounded evidence bundle for each request. keep generated summaries and embeddings optional. neither should be required to capture or read history.

what the existing tests establish

at 100,000 synthetic events, persistent MCP resume took 0.116 ms at the median and selective search took 0.650 ms. CLI resume took 3.276 ms; common-term search took 60.672 ms, with a 74.292 ms p95. unchanged refresh took 52.402 ms. these are prior recorded results, checked against the repository reports, not new runs for this research.

the Claude Code handoff used nine MCP calls and took 22.307 seconds. Codex used thirteen calls and took 41.304 seconds. this gap makes fewer agent round trips a useful experiment. it does not establish how much time came from model inference, scheduling, network, or tool use.

the decision in one view

1. first: stronger handoff tests and source/state relationships.
2. next: query-aware evidence bundles, compact tool results and batched reads.
3. then: improve lexical recall, broad-query scaling and repeated imports.
4. conditional: local embeddings and source-linked summaries.
5. defer: general graph extraction, autonomous memory rewriting and a separate vector service.

local evidence: docs/benchmark-100k.json; docs/harness-verification.json; src/store.rs; src/import.rs; src/schema.sql.

limits: the benchmark uses repetitive synthetic history, one task and 50 warm samples per query. it does not prove cold performance, multi-task scaling, semantic recall or token savings at equal task success. the live checks are smoke tests; some assertions check strings rather than the meaning of the recovered state.

what the literature changes

retrieve evidence, not just summaries

[LongMemEval](#) separates indexing, retrieval and reading across 500 questions. its experiments favor turn-sized units and fact-enriched search keys, while replacing original evidence with summaries or facts can lose useful information. its reported gains use a dense retriever and specific readers, so they are not expected FTS5 gains. for SQnic, enrich the index but retain links to original events. time-aware retrieval also deserves a dedicated test.

summary quality is only half the problem

[SWE Context Bench](#) evaluated 99 related coding tasks under five conditions. reported resolution was 26.26% without experience, 22.22% with autonomously selected summaries and 34.34% with oracle-selected summaries. a good summary can help when the right one is supplied. the autonomous result warns against making summary retrieval the only path. the tasks include shared resolving PRs, and the reported resolution metric requires bug-fixing tests, with regression tests reported separately. SQnic should require both.

try cheap output reduction before model compression

[The Complexity Trap](#) compares masking old tool observations with model summaries on 500 SWE-bench Verified tasks. its v2 table reports Qwen3-Coder masking at 54.8% solved and \$0.61 per task, versus summaries at 53.8% and \$0.64. solve-rate uncertainty overlaps. some other model settings lose accuracy under compression. this supports a cheap baseline: short tool-result headers with links to the complete output. the study concerns active context management, not cross-harness memory.

recent ideas worth adapting selectively

[LightMem](#) batches and defers memory consolidation. its large token-reduction headline concerns memory construction/update overhead, excluding retrieval and QA. use episode-boundary enrichment outside SQnic's query path; do not transfer its headline to our latency.

[SimpleMem](#) combines normalized facts with semantic, lexical and metadata retrieval. borrow names, dates and source-linked search keys. its conversational QA results do not prove code identifier or commit accuracy, and its semantic compression is not reversible storage compression.

[LeanMem](#), an august 2026 preprint, separates compact facts, changing events and source-grounded records. this is a useful design hypothesis for constraints, activity and tool evidence. its personal-memory experiments and new preprint status leave coding transfer unverified.

proposed local architecture

all items on this page are proposals, not implemented behavior.

capture → normalize → retrieve → expand

capture exact source records in SQLite. normalize harness-specific envelopes into typed event metadata. group related events into an episode. retrieve current state and matching episodes. expand the original evidence only where it supports the request.

1. preserve relationships before generating knowledge

add optional session ID, parent event, tool-call ID, source timestamp and ingestion timestamp. preserve unknown fields in raw records. a tool result must link to its call; a fork must keep its parent. missing metadata remains unknown, rather than being guessed from nearby text.

use small SQLite relations for episode membership, supersession and evidence links. retain canonical repository/worktree identity. make current-state and historical-as-of queries explicit. later import order must not silently imply later real-world state. records on another branch must not supersede the active branch without an explicit relationship.

2. return a bounded evidence bundle

extend resume with optional query and scope, keeping the existing default behavior compatible. return current constraints, relevant decisions, tool-result headers, Git state and original evidence IDs. use lexical matches and explicit metadata first. deduplicate overlapping excerpts and reserve space for required constraints before lower-priority history.

apply a response budget to the whole result, including metadata. bytes divided by four is an estimate, not a token guarantee. expose the estimate and omissions; allow an optional tokenizer when a harness requires an exact model-specific budget. never cut a constraint into a misleading partial sentence.

add bounded read-many for event IDs and commit references. cap item count and total output. each item has a typed result or error, so one missing record cannot be mistaken for complete evidence. measure whether this removes sequential agent turns rather than simply returning more text.

3. make commit context verifiable

record the observed HEAD and dirty-worktree state at capture/checkpoint time. store explicit evidence links from decisions and test runs to a revision or worktree snapshot. a nearby timestamp does not prove that a test validated a commit.

keep deterministic Git facts separate from agent-written explanations. an optional commit summary should state changed behavior, rationale, evidence IDs and unresolved risks. mark unknown rationale or test coverage as unknown. show ancestry/reachability on the selected branch; retain other indexed history without presenting it as current. Git supplies revision and path filters, not the reason a change was made. [Git documentation](#)

harness integration

keep CLI and persistent stdio MCP as equivalent interfaces. a short skill explains capture, query, expansion and checkpoint use. automatic capture needs a small adapter for each harness's available export or lifecycle hook. a universal skill cannot expose history that a harness does not make available. test automatic discovery separately from a prompt that explicitly names the tools.

performance and dependency choices

broad lexical queries: measure the actual scope

SQnic already uses ORDER BY rank with LIMIT, the FTS5 pattern documented for efficient ranked results. replacing it with explicit bm25 sorting is not a new optimization. prefix indexes target prefix searches, not ordinary common-word queries. reducing FTS detail can remove useful features; it needs a separate quality and storage test. [SQLite FTS5](#)

test 1, 10 and 100 tasks at fixed total size, then increase total size. compare selective identifiers, common words, absent words and multiple terms. the current global FTS index joins back to a task-filtered event table. profile that path before choosing an indexed task field or partitioning. never limit global candidates before task filtering: that can discard valid task results.

imports: preserve the integrity contract

src/import.rs rereads the consumed prefix to verify its SHA-256 before appending. this explains why an unchanged refresh still scales with source bytes. skipping that read changes the guarantee. file size and modification time alone cannot prove that an old prefix is unchanged.

first coalesce refresh requests and measure bytes read. if frequent capture becomes a bottleneck, add an explicit append-only ingestion interface with idempotency keys, or SQnic-owned sealed chunks. keep strict verification for arbitrary external files. a Merkle tree cannot identify arbitrary edits in unread external bytes without a trusted change signal.

disk compression and prompt compression are separate

changing Markdown to a binary format does not itself reduce tokens sent to an agent. selection and representation control prompt size. lossless compression can reduce disk use, but adds decode work and random-access design choices. test independently compressed raw-record chunks only if actual disk growth matters. keep the searchable representation separate and verify exact byte round trips. do not add a compressor based on synthetic compression ratios alone.

optional semantic search

first test episode search, explicit aliases and separate exact-identifier matching against current strict term matching. then compare a small local embedding model plus rank fusion with lexical retrieval. record model size, cold start, resident memory, index time and paraphrase accuracy. preserve exact matches and task filtering. add a vector index only if brute-force search over the measured corpus misses a latency target.

[engram-mcp](#) is a useful Rust/SQLite/local-embedding implementation reference. [episodic-memory](#) is useful for session import and search integration. neither establishes that SQnic needs the same dependencies or that its latency results are directly comparable.

[HippoRAG2](#) supports graph-assisted multi-hop retrieval, but its evaluated extraction/filtering and embedding models are far heavier than this tool's core. first test task → decision → commit → test links in SQLite. consider learned graph expansion only after the evaluation identifies persistent multi-hop failures.

evaluation that can guide the next release

use a coding-handoff pilot

proposed corpus: 12 development episodes and 24 frozen held-out episodes from at least four small repositories. each held-out episode has four factual probes and one executable continuation task. keep each task's entire repository/conversation lineage in one split. this is a pilot, not enough for broad product claims.

include exact identifiers, paraphrased decisions, corrected constraints, failed approaches, missing evidence, multi-event explanations, dirty files, branch changes and test results tied to an older revision. add harmless adversarial instructions inside imported outputs and commit messages. imported history remains evidence, not new authority to act.

checkpoint each episode at time t . expose only source history and Git objects available at t . keep future fixes, hidden tests and expected answers outside the retrieval backend. compare no memory, full available history, the existing Markdown-notes workflow, current SQnic and oracle evidence. keep the reader model and harness settings fixed within each paired comparison.

score three separate layers

retrieval: required-evidence recall at 1k, 2k, 4k and 8k token budgets; complete-evidence success; duplicate tokens; correct source IDs; first-call evidence coverage.

reasoning: structured answers with values, source IDs, revision IDs and abstention fields. explicitly reject superseded values. manually assess whether cited evidence supports the answer. marker presence alone is not a semantic pass.

continuation: run both bug-fixing and regression tests in disposable checkouts. record task success, constraint violations, stale claims, tool calls, wall time and model usage. include note-writing and index/enrichment cost. report time and tokens per successful task as well as per attempt.

borrow protocols, not an entire research stack

[LongMemEval-V2](#) provides a closer agent-memory framing: 451 questions, trajectory histories, compact evidence to a fixed reader, and an accuracy-latency frontier. its environment-knowledge questions still differ from commit-aware coding continuation.

[ContextBench](#) tracks retrieved code context, precision/recall and repeated retrieval across coding trajectories. adapt its distinction between evidence accessed and evidence used. a reference evidence set need not include every valid alternative solution.

realistic runtime checks

measure warm and cold process/cache conditions separately; p50/p95/p99; peak RSS; database size; import throughput; refresh bytes read; mixed reader/writer contention. use varied event sizes and real exported shapes, not only repeated text. increase repetitions before making tail-latency claims.

live-test both Claude Code → Codex and Codex → Claude Code on a fixed representative subset, with three repeats per condition. record exact versions and paired per-case outcomes. add Pi and Cursor export paths as separate compatibility checks. report unsupported capture surfaces explicitly.

ranked implementation queue

stage 1: establish correctness

strengthen the current harness test to assert current values, superseded values, evidence IDs and revision support. add temporal, branch and absent-evidence fixtures. then normalize tool/session relationships and record checkpoint Git state. acceptance: every deterministic source, chronology and task-isolation assertion passes. no corrupt import or unsupported test claim is accepted.

stage 2: reduce agent work

add compact tool-result headers, query-aware resume and bounded read-many. keep raw expansion available. compare each addition separately with the current release. acceptance: fewer calls or tokens on paired tasks, with no observed continuation regression; report uncertainty rather than claiming equivalence from a small sample. a 30% reduction can be a product target, not a literature-backed promise.

stage 3: improve retrieval and scaling

test episode grouping and richer lexical keys. profile broad searches across tasks and frequent imports. select an index or ingestion change only when its measured gain justifies extra code and its correctness contract remains explicit. log database timings separately from total harness time.

stage 4: add intelligence only where it helps

run optional local embeddings on the held-out paraphrase cases. test agent-written episode/commit summaries against compact raw evidence. charge construction time and tokens to the comparison. keep each derived item versioned and source-linked. if simpler retrieval reaches the same quality/cost frontier, stop there.

a lean implementation boundary

use the existing Rust/SQLite core and small modules for adapters, normalization, retrieval and presentation. use SQL relations for known links. avoid a background service, graph database, model scheduler or automatic rewrite engine until a measured requirement needs one. an optional enrichment worker may run through the user's chosen harness; its output is derived data, never a replacement for captured records.

this design is an engineering synthesis of the evidence, not a reproduced result from any paper. no implementation files were changed and no new harness or performance benchmark was run during this research.

source and research notes

primary sources reviewed: LongMemEval (Wu et al., ICLR 2025, v2); The Complexity Trap (Lindenbauer et al., september 2025 v2); LightMem (Fang et al., october 2025 v1); SimpleMem (Liu et al., january 2026 v1); SWE Context Bench (Zhu et al., february 2026 v1); ContextBench (Li et al., february 2026 v3); LongMemEval-V2 (Wu et al., may 2026 repository/preprint); LeanMem (Liao et al., august 2026 v1); HippoRAG2 (Gutiérrez et al., 2025 v2). clickable paper and repository links appear beside the relevant findings.

official SQLite and Git documentation and first-party repository pages were checked on 5 september 2026. research focused on original papers, disconfirming coding results and the current SQnic code. quantitative findings use their stated populations and measurement boundaries. implementations were inspected as references, not installed or benchmarked against SQnic. newer preprint gains remain independently unverified. the evidence is sufficient to choose the first experiments; it does not settle the eventual value of semantic or graph retrieval.